==========

APEX RETAIL GROUP | ANALYTICS TRAINING PROGRAM
DOCUMENT: CLASS NOTES (SESSION 7)
TOPIC: RELATIONAL WRANGLING — MERGING & RESHAPING DATASETS
LEVEL: INTERMEDIATE (DAY 7)

==========

1. CONTINUITY & BUSINESS SCENARIO RECAP

In Session 6, you learned how to uncover performance drivers by grouping, aggregating, and segmenting clean tabular datasets using `.groupby()` and multi-metric `.agg()`.

Now, in your seventh week at Apex Retail Group:

Liam Vance, Marketing Manager, approaches your desk with an urgent request:

> **"Our team recently executed two major promotional campaigns (Campaign Alpha and Campaign Beta) to drive customer acquisition. Finance wants to know if customers who participated in Campaign Alpha actually spent more per order than non-campaign customers.**
>
> **The problem is that our data is scattered across three separate files: transaction logs in one table, customer demographic profiles in another, and campaign participation logs in a third.**
>
> **We cannot analyze campaign ROI or customer tier performance until these tables are linked together properly. Can you wrangle and merge these relational datasets and evaluate whether Campaign Alpha successfully drove higher basket sizes?"**

In this session, you step beyond single-table operations. You will learn how relational databases work, how to connect separate datasets using keys (`pd.merge()`), how to stack datasets (`pd.concat()`), and how to reshape data for executive reporting (`pivot_table()`).

2. THE REALITY OF RELATIONAL DATA & JOIN KEYS

In modern organizations, data is rarely stored in a single massive table. To avoid redundancy, databases normalize information into separate tables:
- Table A (Transactions): Stores order ID, customer ID, date, and revenue.
- Table B (Customers): Stores customer ID, signup date, and demographic tier.
- Table C (Campaigns): Stores customer ID and campaign code.

To analyze this data, an analyst must perform **Relational Joins** using common identifying columns known as **Keys** (e.g., `customer_id`).

3. PANDAS RELATIONAL WRANGLING TOOLKIT

A. Joining Tables (`pd.merge()`)

Pandas `merge()` combines two DataFrames based on a shared key column:

- **Inner Join** (`how='inner'`): Keeps only rows where the key exists in BOTH tables.
- **Left Join** (`how='left'`): Keeps ALL rows from the left table, matching rows from the right table where available (unmatched right rows become `NaN`).
- **Right Join** (`how='right'`): Keeps ALL rows from the right table, matching rows from the left.
- **Outer Join** (`how='outer'`): Keeps ALL rows from both tables, filling missing matches with `NaN`.

```
        combined_df = pd.merge(df_transactions, df_customers,
on="customer_id", how="left")
```

B. Concatenating DataFrames (`pd.concat()`)

When you have data split across files with identical columns (e.g., Q1 sales and Q2 sales), use `concat()` to stack them vertically or horizontally:

```
        annual_sales = pd.concat([df_q1, df_q2], ignore_index=True)
```

C. Reshaping with Pivot Tables (`pd.pivot_table()`)

To transform long-format transactional records into wide, executive-ready summary matrices:

```
        pivot_summary = pd.pivot_table(
            df,
            values="revenue",
            index="customer_tier",
            columns="campaign",
            aggfunc="mean"
        )
```

4. EXECUTING THE CAMPAIGN MERGE & ANALYSIS IN PANDAS

Below is the clean, annotated Python code required for Liam's marketing campaign audit:

```python

import pandas as pd

# Step 1: Load separate relational tables
transactions_data = {
    "order_id": ["ORD-901", "ORD-902", "ORD-903", "ORD-904", "ORD-905"],
    "customer_id": ["CUST-01", "CUST-02", "CUST-03", "CUST-04", "CUST-05"],
    "order_revenue": [250.00, 89.00, 450.00, 120.00, 600.00]
}
df_txns = pd.DataFrame(transactions_data)

customers_data = {
    "customer_id": ["CUST-01", "CUST-02", "CUST-03", "CUST-04", "CUST-06"],
# CUST-05 missing profile, CUST-06 extra profile
    "customer_tier": ["Gold", "Silver", "Platinum", "Gold", "Silver"],
    "signup_year": [2024, 2025, 2023, 2025, 2024]
}
df_custs = pd.DataFrame(customers_data)

campaign_data = {
    "customer_id": ["CUST-01", "CUST-02", "CUST-03", "CUST-05"],
    "campaign_name": ["Campaign_Alpha", "Campaign_Beta", "Campaign_Alpha",
"Campaign_Alpha"]
}
df_camp = pd.DataFrame(campaign_data)

# Step 2: Merge transactions with customer profiles using a Left Join
# Left join ensures we retain all transactions even if a customer profile
is missing
df_merged = pd.merge(df_txns, df_custs, on="customer_id", how="left")

# Step 3: Merge campaign participation log using a Left Join
df_full = pd.merge(df_merged, df_camp, on="customer_id", how="left")
```

```
# Step 4: Fill missing campaign tags with "Non-Campaign" for unassigned customers
df_full["campaign_name"] = df_full["campaign_name"].fillna("Non-Campaign")

print("--- FULLY MERGED DATASET ---")
print(df_full)

# Step 5: Aggregate order revenue by campaign using groupby
campaign_performance = df_full.groupby("campaign_name").agg(
    total_revenue=("order_revenue", "sum"),
    avg_order_value=("order_revenue", "mean"),
    order_count=("order_id", "count")
)

print("\n--- CAMPAIGN PERFORMANCE SUMMARY ---")
print(campaign_performance)
```

INTERPRETING NUMBERS & FORMULATING BUSINESS RECOMMENDATIONS

A. Observation (The Merged Analysis):

- Campaign Alpha Revenue: Generated $1,300.00 across 3 orders with an average order value (AOV) of $433.33.
- Campaign Beta Revenue: Generated $89.00 across 1 order with an AOV of $89.00.
- Non-Campaign Revenue: Generated $120.00 across 1 order with an AOV of $120.00.

B. Interpretation & Business Meaning:

- Campaign Alpha substantially outperforms both Campaign Beta and organic orders, driving a much higher average basket size ($433.33 vs $120.00).
- Relational joining allowed us to connect customer IDs across disparate files without losing transaction integrity or corrupting financial sums.

C. Professional Recommendation to Liam Vance (Marketing Manager):

"By linking our transaction logs with customer and campaign files, we found that Campaign Alpha successfully drives high-value orders, averaging $433.33 per transaction compared to $120.00 for non-campaign orders.

Recommendation: Reallocate remaining marketing budget from Campaign Beta to scale Campaign Alpha across Platinum and Gold customer tiers."

CORE MINDSET HABITS REINFORCED TODAY

1. Verify join keys before merging: Always check that your key columns (`customer_id`) share identical data types and casing before executing `pd.merge()`.
2. Choose the correct join type: Understand whether a Left Join or Inner Join is required so you do not accidentally drop valid financial transactions.